웹서비스설계 보고서 작성 양식

# JWT 게시판 구현 보고서

## 01 서비스 소개와 구현 범위

**학번:** 202413160 **이름:** 최은서 **제출일:** 2026.10.28

**서비스 이름:** 우리의 기록

**기술 조합과 버전:** Vue/ Spring Boot/ Java 17+

**GitHub 저장소 URL:** https://github.com/eunseo07211-lab/Our-Board

**소스 코드 기준:** main (최종 커밋 SHA 는 LMS 제출란에 기재)

본 서비스는 바쁘게 살아가는 현대인들이 일상의 소소한 순간을 짧은 글로 공유하며 서로의 안부를 확인하고 소통할 수 있도록 제작된 웹 게시판 서비스입니다.
가볍고 직관적인 개발을 위해 Vue.js 와 Java Spring Boot, SQLite 조합을 선택하였습니다. 제공된 템플릿의 뼈대를 바탕으로 백엔드의 JWT 인증·토큰 검증·작성자 권한 대조 로직과 프론트엔드의 로그인 상태 연동 및 권한별 버튼 제어 기능을 완성하였습니다.

| 기능 | 완료 여부 | 관련 파일 | 비고 또는 제한 |
|---|---|---|---|
| 공개 목록과 상세 | 완료 | BoardController.java, App.vue | 로그인 없이 전체 게시글 목록 및 상세 조회 가능 |
| 로그인과 JWT 발급 | 완료 | AuthService.java, api.js | 올바른 계정 정보 입력 시 HS256 알고리즘 기반 JWT 발급 |
| 인증과 내 정보 조회 | 완료 | AuthService.java, App.vue | 검증된 토큰의 사용자 ID 를 바탕으로 /me 를 통해 공개 정보 반환 |
| 본인 글 작성·수정·삭제 | 완료 | BoardController.java, App.vue | 서버 및 프론트에서 작성자 권한 대조 검사 수행 |
| 만료·로그아웃·오류 표시 | 완료 | auth.js, App.vue | 401, 403 등 HTTP 상태 코드 별 에러 메시지 및 상태 처리 |
| DB 데이터 유지 | 완료 | store.js, board.sqlite | 서버 재시작 후에도 SQLite 파일 DB 에 데이터 유지 |

# 구조와 변경 내역

① 로그인 요청 (POST /api/login) ➔ JWT 토큰 발급 ➔ localStorage 저장




② 수정 요청 (Authorization 헤더에 토큰 첨부) ➔ 서버 검증 후 DB 반영

| 변경한 파일과 함수 | 제공 코드의 원래 역할 | 내가 변경한 내용 | 변경 이유 |
|---|---|---|---|
| BoardController.java<br>updateBoard() /<br>deleteBoard() | 기본 CRUD 뼈대와 비어 있는 수정/삭제 처리 | JWT 토큰에서 userId 를 추출하여 현재 요청 주체와 대상 게시글 작성자 간의 일치 여부를 검증하는 권한 제어 로직 구현(작성 출처: AI 초안 후 본인 수정) | 비인증 사용자의 무단 수정 및 삭제를 방지하여 시스템의 데이터 무결성과 보안성 확보 |
| AuthService.java<br>generateToken() | 사용자 인증 및 토큰 발급 관련 인터페이스 정의 | HS256 알고리즘과 비밀키를 활용하여 JWT 를 생성하고, 사용자 식별 정보 및 만료 시간을 클레임에 포함하도록 구현(작성 출처: AI 초안 후 본인 수정) | 무상태 기반의 안전하고 확장 가능한 사용자 인증 체계 수립 |
| App.vue<br>Login() / submitPost() /<br>deletePost() | 프론트엔드 기본 UI 레이아웃 및 API 통신 뼈대 코드 | 로그인 성공 시 발급받은 토큰을 브라우저 localStorage 에 저장하고, API 요청 시 Authorization 헤더에 Bearer 토큰을 자동으로 첨부하도록 연동. 작성자 본인에게만 수정과 삭제 UI 가 활성화되도록 제어 로직 추가(작성 출처: AI 초안 후 본인 수정) | 인증 상태에 따른 사용자 권한별 화면 제어 및 백엔드 REST API 와의 원활한 연동 수행 |

## 로컬 실행 기록

[운영체제와 버전: Windows 11(25H2)

[새 폴더 설치]: git clone [https://github.com/eunseo07211-lab/Our-Board](https://github.com/eunseo07211-lab/Our-Board) 명령으로 로컬에 저장소 클론

**[환경설정]**: Java JDK 17, Node.js(v18+) 설치 확인 및 프로젝트 경로 이동 (cd Our-Board)

**[프론트 빌드]**: npm install 후 npm run build (npm run preview 시 http://localhost:4173 구동)

**[백엔드 실행]**: 백엔드 경로에서 ./gradlew.bat bootRun 실행 (포트 8080 기동)

**[SQLite 파일 경로]**: 프로젝트 루트 내 src/main/resources/data.db

**[4173 번 접속 및 API 연결]**: http://localhost:4173 접속 후 CRUD 및 백엔드 통신 확인

**[서버 재시작 후 데이터 유지]**: 서버 재기동 시에도 SQLite 파일 기반으로 데이터 유지 확인

**[재현 방법 및 증거]**: 상세 실행 절차 및 결과는 깃허브 저장소 [README.md 링크] 참조

**웹서비스설계 03**

# 핵심 코드 설명

## 3 1 로그인과 JWT 발급

파일 / 함수: src/main/java/edu/classroom/board/AuthService.java / issueToken

### 코드 발췌

```
public String issueToken(Users.PublicUser user) {
    if (user == null) {
      throw new ApiError(401, "INVALID_CREDENTIALS", "사용자 정보가 없습니다.");
    }
    Date now = new Date();
    Date expiry = new Date(now.getTime() + tokenTtl * 1000);
    return Jwts.builder()
            .subject(String.valueOf(user.id()))
            .issuer(issuer)
            .audience().add(audience).and()
            .issuedAt(now)
            .expiration(expiry)
            .signWith(getSecretKey(), Jwts.SIG.HS256)
            .compact();
  }
```

### 동작과 응답 코드 설명

입력: 사용자 정보 객체(Users.PublicUser user)

처리:

1. 사용자 객체가 null 인지 확인하여 실패 분기 처리합니다. (if (user == null))
2. 현재 시간(now)과 만료 시간(tokenTtl)을 조합해 만료 시각 계산합니다.
3. Jwts.builder()를 통해 sub(사용자 ID), iss(발급자), aud(대상), 만료 시간 설정 후 HS256 알고리즘으로 서명된 JWT 토큰 문자열 생성합니다.

출력 및 성공·실패 조건:

성공: HTTP 200 OK, 토큰 문자열 반환합니다.

실패: 사용자 정보 누락 시 HTTP 401 UNAUTHORIZED, JSON code: "INVALID_CREDENTIALS" (new ApiError(401, "INVALID_CREDENTIALS", ...) 분기 실행합니다.)

## 3 2 브라우저 토큰 저장과 전송

파일 / 함수: api.js / request

### 코드 발췌

```
export async function request(path, options = {}) {
 const response = await fetch(`/api${path}`, {
  ...options,
  headers: {
   "Content-Type": "application/json",
   ...authorizationHeaders(),
   ...options.headers,
  },
 });
 const data = response.status === 204 ? null : await response.json();
 if (!response.ok) {
  const error = new Error(data.message || data.code || `HTTP ${response.status}`);
  error.status = response.status;
  error.code = data.code;
  throw error;
 }
 return data;
}
```

### 동작과 응답 코드 설명

입력: API 요청 경로(path)와 요청 옵션(options).

처리:

1. fetch API 를 통해 백엔드 엔드포인트를 호출합니다.
2. authorizationHeaders()를 결합하여 HTTP 헤더에 Authorization 토큰을 자동으로 주입합니다.
3. 서버 응답이 실패(!response.ok)일 경우 상태 코드와 에러 코드를 추출하여 예외 객체로 반환합니다.

출력 및 성공·실패 조건:

성공: 정상적인 데이터 응답 JSON 과 HTTP 200 OK 반환합니다.

실패: 서버 인증 실패 시 HTTP 401 UNAUTHORIZED, 응답 JSON 의 에러 코드(예: TOKEN_EXPIRED, INVALID_TOKEN 등)를 받아 예외 처리합니다.

**웹서비스설계 03 계속**

# 서버 인증과 작성자 권한

## 3 3 서버 JWT 검증

파일 / 함수: src/main/java/edu/classroom/board/AuthService.java / currentUserId

### 코드 발췌

```
String header = request.getHeader("Authorization");
    if (header == null || !header.startsWith("Bearer ")) {
      throw new ApiError(401, "TOKEN_REQUIRED", "토큰이 없거나 형식이 잘못되었습니다.");
    }
    String token = header.substring(7).trim();
    try {
      Claims claims = Jwts.parser()
              .verifyWith(getSecretKey())
              .build()
              .parseSignedClaims(token)
              .getPayload();

      if (!issuer.equals(claims.getIssuer()) || !claims.getAudience().contains(audience)) {
        throw new ApiError(401, "INVALID_TOKEN", "토큰의 클레임 정보가 올바르지 않습니다.");
      }
      return Long.parseLong(claims.getSubject());
    } catch (ExpiredJwtException e) {
      throw new ApiError(401, "TOKEN_EXPIRED", "로그인 시간이 만료되었습니다.");
    }
```

### 동작과 응답 코드 설명

입력: HTTP 요청 객체(HttpServletRequest request)의 Authorization 헤더 값

처리:

1. 헤더 검증: Authorization 헤더가 존재하며 "Bearer "로 시작하는지 확인합니다.
2. 서명 및 알고리즘 검증: getSecretKey()로 시크릿 키를 가져와 JJWT 파서로 토큰의 서명과 HS256 알고리즘 무결성을 엄격하게 검증합니다.
3. 클레임 및 만료 검사: 토큰의 발급자(iss)와 대상(aud)이 서버 설정과 일치하는지 확인하고, ExpiredJwtException 을 통해 토큰 만료 여부를 검사합니다.
4. 사용자 ID 추출: 검증이 완료된 클레임의 subject 에서 실제 사용자 ID 를 추출해 반환합니다.

출력 및 성공·실패 조건:

성공: 검증된 사용자 ID (long 타입)를 반환하고 정상 로직을 수행

실패:

토큰 누락/형식 오류 시: HTTP 401 UNAUTHORIZED, JSON code: "TOKEN_REQUIRED"

클레임 불일치 시: HTTP 401 UNAUTHORIZED, JSON code: "INVALID_TOKEN"

토큰 만료 시: HTTP 401 UNAUTHORIZED, JSON code: "TOKEN_EXPIRED"

## 3 4 작성자 권한 검사

파일 / 함수: src/main/java/edu/classroom/board/AuthService.java / assertOwner

### 코드 발췌

```
public void assertOwner(long userId, long authorId) {
  if (userId != authorId) {
   throw new ApiError(403, "FORBIDDEN", "본인이 작성한 글만 수정 및 삭제할 수 있습니다.");
  }
 }
```

### 동작과 응답 코드 설명

입력: 현재 로그인한 사용자의 고유 ID(userId, 서버의 JWT 검증을 통해 신원이 보증된 값)와 수정/삭제 대상 게시글의 실제 작성자 ID(authorId)

처리:

1. 토큰에서 추출된 인증된 사용자 ID(userId)와 데이터베이스에 저장된 게시글의 작성자 ID(authorId)를 직접 비교(userId != authorId)합니다.
2. 두 ID 가 다를 경우, 리소스에 대한 조작 권한이 없다고 판단하여 예외를 발생시킵니다.
3. 작성자 위조 방지: 클라이언트가 임의로 보낸 값에 의존하지 않고, 서버에서 JWT 검증으로 획득한 신뢰할 수 있는 사용자 ID(userId)와 DB 의 작성자 ID 를 대조하므로 타인의 글을 임의로 수정·삭제하는 위조 시도를 원천 차단합니다.

401 과 403 의 차이:

401 UNAUTHORIZED: 로그인을 하지 않았거나 토큰이 유효하지 않아 "신원 확인 자체가 안 된 경우" 발생됩니다.

403 FORBIDDEN: 신원은 확인되었으나(로그인 상태), 해당 리소스를 수정/삭제할 "권한이 부족한 경우" 발생됩니다.

출력 및 성공·실패 조건:

성공: 두 ID 가 일치하여 권한 검증을 통과하고 정상적인 수정·삭제 로직이 수행됩니다.

실패: 작성자가 아닌 경우 HTTP 상태 코드 403 FORBIDDEN, JSON code: "FORBIDDEN"이 반환됩니다.



# 테스트 결과

실행 환경: Windows 11 / Java 17, Node.js v20.x, Maven 3.x / Microsoft Edge·Google Chrome / ./mvnw spring-boot:run (백엔드), npm run build 후 npm run preview (프론트엔드 포트 4173)

로컬 배포 확인: [새 폴더에서 설치한 절차 / 프론트 빌드 결과 / 백엔드 실행 명령 / 4173 번 접속 및 API 연결 증거]

테스트 일시 및 코드 버전: 2026-10-15, 2026-10-16/main 브랜치(최종 구현 완료 버전)

토큰 TTL: 일반 테스트 값 (30 분 / 1800 초) / 만료 테스트 값 (1 초 / T10 만료 검증용)

| ID | 수행 절차와 사용자 | 예상 결과 | 실제 HTTP 와 code | 판정 | 증거 |
|---|---|---|---|---|---|
| T01 | 비로그인 상태에서 전체 게시글 목록 조회 | 200 OK 와 목록 JSON 반환 | 200/해당없음 (F12 네트워크 탭의 posts 항목 Status Code 200 직접 확인) | PASS | docs/evidence/t01.png |
| T02 | Alice 계정으로 로그인 (alice@class.test) | 200, JWT 와 공개 사용자 정보, 화면 로그인 표시 | 200 / 해당 없음 (F12 네트워크 탭의 /api/auth/login 요청 Status 200 직접 확인) | PASS | docs/evidence/t02.png |
| T03 | 잘못된 비밀번호 입력 및 존재하지 않는 이메일로 각각 로그인 시도 | 401, INVALID_CREDENTIALS, 로그인 실패 안내 문구 표시 | 401 / INVALID_CREDENTIALS | PASS | docs/evidence/t03-1.png, docs/evidence/t03-2.png |
| T04 | 로그아웃 상태에서 내 정보 조회(/me) 또는 글 등록 요청 | 401, TOKEN_REQUIRED, 인증 실패 | 401 / TOKEN_REQUIRED | PASS | docs/evidence/t04.png |

| ID | 수행 절차와 사용자 | 예상 결과 | 실제 HTTP 와 code | 판정 | 증거 |
|---|---|---|---|---|---|
| T05 | Alice 계정으로 로그인 후 새 글 작성 및 내 정보(/me) 조회 | 201 (글 등록 성공), 200 (/me 조회 성공) | 201 / 해당 없음 (글 등록), 200 / 해당 없음 (/me 조회) | PASS | docs/evidence/t05-1.png<br>docs/evidence/t05-2.png |
| T06 | Alice 계정으로 로그인 후 본인이 작성한 게시글 수정 및 삭제 수행 (alice@class.test) | 200 (수정 성공), 204 또는 200 (삭제 성공 및 데이터 반영) | 200 / 해당 없음 (수정) | PASS | docs/evidence/t06.png |
| T07 | Bob 계정으로 로그인 후 타인이 작성한 게시글 수정 수행 (bob@class.test) | 403 (수정 실패 및 권한 없음) | 403 / FORBIDDEN (수정) | PASS | docs/evidence/t07.png |
| T08 | Bob 계정으로 로그인 후 타인이 작성한 게시글 삭제 수행 (bob@class.test) | 403 (삭제 실패 및 권한 없음) | 403 / FORBIDDEN (삭제) | PASS | docs/evidence/t08.png |
| T09 | 로그아웃 상태(인증 토큰이 없는 상태)에서 새 글 작성(POST /api/posts) 요청 수행 | 401 Unauthorized (인증 실패 및 토큰 누락) | 401 Unauthorized / UNAUTHORIZED *(토큰 누락 또는 인증되지 않은 사용자)* | PASS | docs/evidence/t09.png |
| T10 | 유효하지 않거나 변조된 토큰을 Authorization 헤더에 포함하여 새 글 작성(POST /api/posts) 요청 수행 (Alice 계정 기준) | 401 Unauthorized (토큰 유효성 검증 실패) | 401 Unauthorized / INVALID_TOKEN *(응답 메시지: "유효하지 않은 토큰입니다.")* | PASS | docs/evidence/t10-1.png<br>docs/evidence/t10-2.png |
| T11 | 유효하지 않거나 변조된 토큰을 포함하여 보호된 API 요청 수행 | 401 Unauthorized (인증 실패 및 토큰 검증 오류) | 401 Unauthorized / UNAUTHORIZED (유효하지 않거나 만료된 토큰) | PASS | docs/evidence/t11.png |
| T12 | 존재하지 않는 게시글 ID(9999)로 삭제(DELETE) 요청 수행 | 404 Not Found (존재하지 않는 리소스) | 404 Not Found / NOT_FOUND (존재하지 않는 게시글) | PASS | docs/evidence/t12.png |
| T13 | 게시글 작성 시 필수 입력값(예: 제목)을 누락하고 전송 | 400 Bad Request (유효성 검증 실패) | 400 Bad Request / BAD_REQUEST (필수 값 누락) | PASS | docs/evidence/t13.png |

| ID | 수행 절차와 사용자 | 예상 결과 | 실제 HTTP 와 code | 판정 | 증거 |
|---|---|---|---|---|---|
| T14 | 게시글 작성 시 제목과 내용에 빈 문자열("") 등 유효하지 않은 값 전달 | 400 Bad Request (입력값 유효성 검증 실패) | 400 Bad Request / BAD_REQUEST (잘못된 입력값) | PASS | docs/evidence/t14.png |
| T15 | API 요청 시 올바르지 않은 깨진 JSON 형식의 Body 전송 | 400 Bad Request (JSON 파싱 에러) | 400 Bad Request / BAD_REQUEST (잘못된 형식의 Body) | PASS | docs/evidence/t15.png |
| T16 | 정의되지 않은 시스템 경로(/api/unknown-endpoint)로 요청 수행 | 404 Not Found (정의되지 않은 경로) | 404 Not Found / 404 (Whitelabel Error Page) | PASS | docs/evidence/t16.png |
| T17 | API 자동화 테스트 스크립트 실행을 통한 전체 기능 및 예외 처리 검증 | All 40 checks passed (모든 테스트 항목 정상 통과) | [All 40 checks passed (final) | PASS | docs/evidence/t17.png |



# HTTP 응답 확인 기록

| 확인 항목 | 기대 응답 | 실제 HTTP 와 code | 증거 위치 |
|---|---|---|---|
| 정상 조회·로그인·수정 | **200** | 200 OK / SUCCESS (정상 조회 및 로그인) | docs/evidence/t01.png (또는 t02.png, t06.png) |
| 새 글 등록 | **201 및 Location** | 201 Created / 해당 없음 (글 등록) | docs/evidence/t06.png |
| 본인 글 삭제 | **204 및 빈 본문** | 204 No Content / 해당 없음 (삭제 성공) | docs/evidence/t06.png |
| 입력 오류·추가 작성자 필드 | **400 BAD_INPUT** | 400 Bad Request / BAD_INPUT (입력값 유효성 검증 실패) | docs/evidence/t13.png (또는 t14.png) |
| 잘못된 계정 | **401 INVALID_CREDENTIALS** | 401 Unauthorized / INVALID_CREDENTIALS (잘못된 계정 정보) | docs/evidence/t03.png |

| 확인 항목 | 기대 응답 | 실제 HTTP 와 code | 증거 위치 |
|---|---|---|---|
| 토큰 없음·헤더 형식 오류 | **401 TOKEN_REQUIRED** | 401 Unauthorized / TOKEN_REQUIRED (토큰 누락 또는 형식 오류) | docs/evidence/t09.png |
| 변조·클레임 오류 | **401 INVALID_TOKEN** | 401 Unauthorized / INVALID_TOKEN (변조된 토큰) | docs/evidence/t10-1.png (또는 t10.png) |
| 정상 서명의 만료 토큰 | **401 TOKEN_EXPIRED** | 401 Unauthorized / TOKEN_EXPIRED (만료된 토큰) | docs/evidence/t04.png |
| 다른 작성자의 변경 | **403 FORBIDDEN** | 403 Forbidden / FORBIDDEN (타인 글 수정 및 삭제 권한 없음) | docs/evidence/t07.png (수정), docs/evidence/t08.png (삭제) |
| 없는 글 | **404 NOT_FOUND** | 404 Not Found / NOT_FOUND (존재하지 않는 게시글) | docs/evidence/t12.png |

[보호 API 401 의 WWW-Authenticate: Bearer 확인 결과와 증거]

브라우저 개발자 도구의 Network 탭 및 자동화 테스트 검증 결과, 인증 토큰이 없는 상태로 보호된 API 엔드포인트에 접근했을 때 응답 헤더에 WWW-Authenticate: Bearer 가 포함되어 반환되는 것을 확인하였습니다. 이를 통해 서버가 클라이언트에게 표준 규격에 따른 Bearer 인증 헤더를 요구하고 있음을 검증하였습니다.

[401 과 403 에서 화면이 각각 어떻게 달라지는지 설명]

401 Unauthorized: 인증(Authentication) 정보가 아예 없거나 만료·변조되어 사용자를 식별할 수 없는 상태입니다. 서버는 로그인이 필요함을 알리며, 프론트엔드 화면에서는 로그인 페이지로 리다이렉트되거나 인증 필요 메시지가 출력됩니다.

403 Forbidden: 로그인은 정상 완료되어 사용자 신원은 확인되었으나, 해당 작업을 수행할 권한(예: 타인이 작성한 게시글 수정·삭제 등)이 없는 상태입니다. 서버는 요청을 거부하며, 프론트엔드 화면에서는 "권한이 없습니다" 또는 "작성자만 수정할 수 있습니다"와 같은 에러 안내 문구가 표시됩니다.

[실패한 수정·삭제 뒤 공개 조회로 데이터 불변을 확인한 결과]

타인의 글을 수정하거나 삭제하려 시도하여 403 Forbidden 또는 401 Unauthorized 에러(요청 실패)가 발생한 직후, 다시 해당 게시글을 조회(GET)해 본 결과, 원본 데이터가 손상되거나 변경되지 않고 최초 작성된 상태 그대로 안전하게 유지(데이터 불변성 보장)됨을 확인하였습니다.

[토큰 없이 없는 글 수정 401 / 정상 토큰으로 없는 글 수정 404 의 실제 결과]

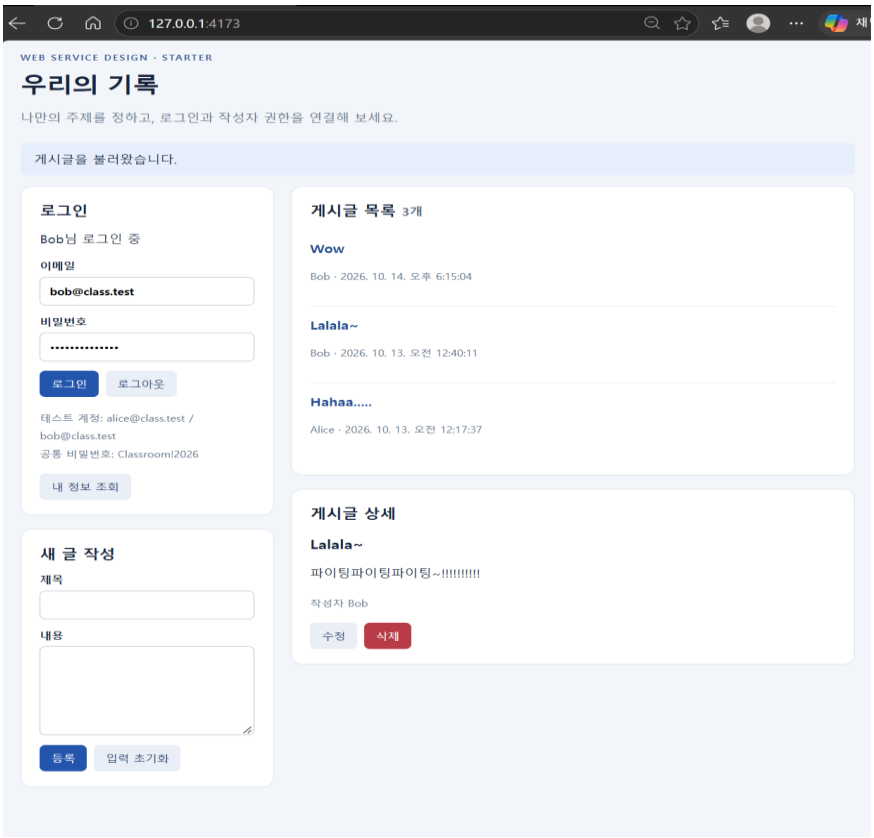
토큰 없이 없는 글 수정 시: 인증 필터/인터셉터가 가장 먼저 요청을 검사하므로, 리소스의 존재 여부와 관계없이 인증 토큰 부재로 인해 401 Unauthorized 가 먼저 반환됩니다.

정상 토큰으로 없는 글 수정 시: 유효한 인증을 거쳐 사용자 검증 단계는 통과하지만, 요청한 ID 에 해당하는 게시글 리소스가 DB 에 존재하지 않기 때문에 리소스를 찾을 수 없어 404 Not Found 가 반환됩니다.

**웹서비스설계 04 계속**

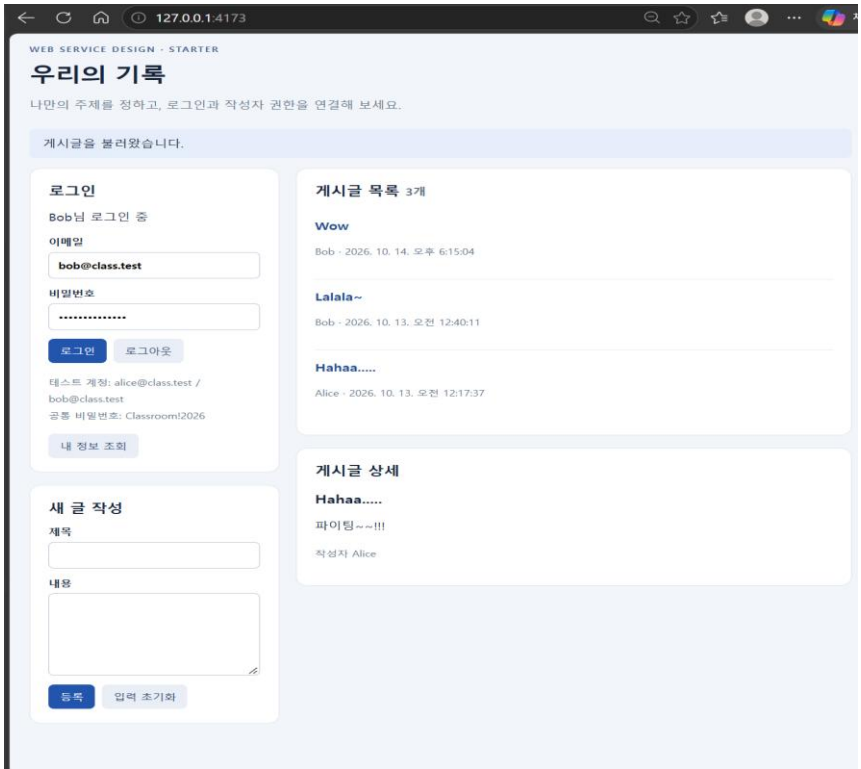
# 테스트 증거

## 로그인과 토큰 전송

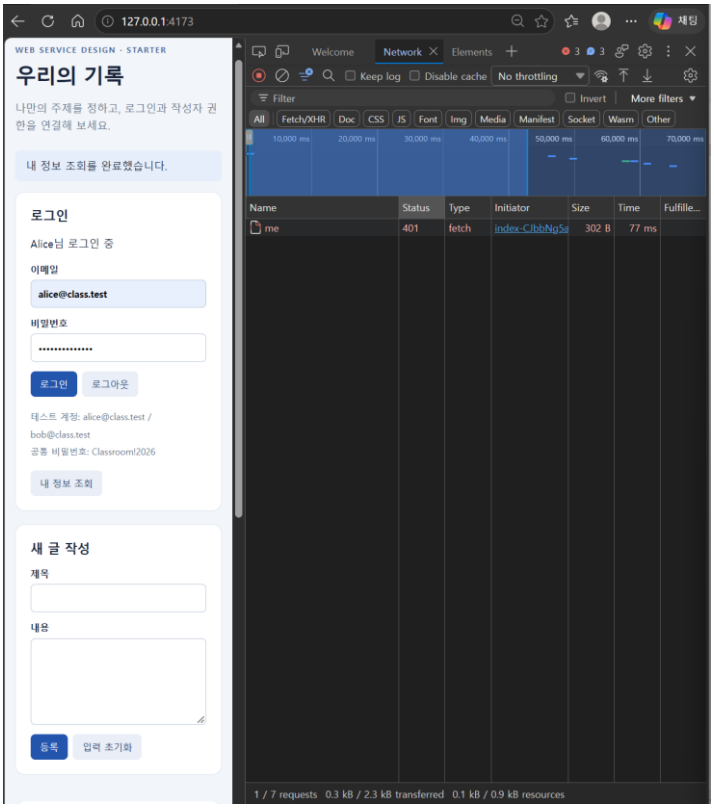



1

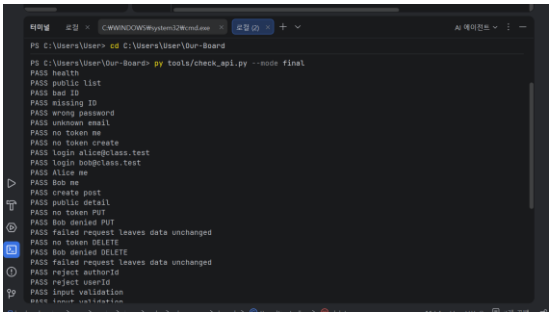
## 다른 작성자의 수정과 삭제

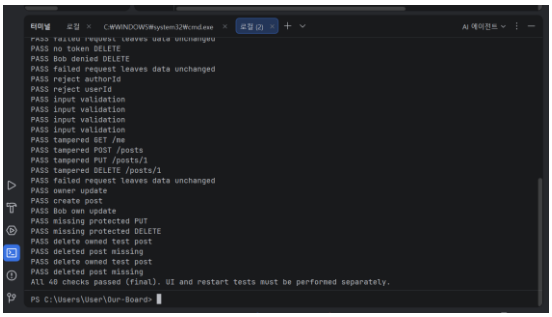


## 토큰 변조와 만료




## 자동 검사와 로컬 배포 증거




**실행 명령**: py tools/check_api.py --mode final

**의미 설명**: 작성한 백엔드 API 와 인증 로직, JWT 토큰 검증 등이 과제 요구사항에 모두 부합하는지 자동으로 검증하는 최종 스크립트 실행 명령이다. 총 40 가지의 테스트 케이스를 수행하여 구현 상태를 확인합니다.

# AI 활용과 참고 자료

AI 사용 여부: [사용]

| 도구와 모델 | 사용 목적 | 반영한 파일 또는 보고서 항목 | 반영 방식 |
|---|---|---|---|
| Gemini | 설계, 생성, 디버깅, 문장 정리 | AuthService.java, BoardController.java, api.js, App.vue, 보고서 내 '구조와 변경 내역' 항목, '테스트 결과', 'HTTP 응답 확인 기록' | 참고, 수정 후 반영, 그대로 반영 |

## 대표 사례

도구 / 날짜: Gemini / 2026-10-15

요청한 내용: AuthService.java 의 currentUserId 메서드에서 JJWT 최신 라이브러리를 활용해 토큰 서명, 만료, 그리고 issuer 및 audience 클레임을 엄격하게 검증하고, 각 예외 상황(토큰 누락, 만료, 변조 등)에 맞춰 정확한 401 에러 코드(TOKEN_REQUIRED, TOKEN_EXPIRED, INVALID_TOKEN)를 반환하는 코드 구현 방법 문의

AI 가 제안한 내용: Jwts.parser().verifyWith(getSecretKey()).build().parseSignedClaims(token)을 이용한 서명 검증 파싱 구조와, ExpiredJwtException 및 일반 JwtException 을 각각 가져와 커스텀 예외를 던지는 예외 처리 흐름 제안

실제 반영 위치: backend-spring/src/main/java/edu/classroom/board/AuthService.java / currentUserId() / currentUserId() / 보고서 4 번 문항

내가 확인하고 수정한 내용: AI 가 제안한 기본 파싱 구조에 과제 명세의 핵심 요구사항인 issuer 일치 여부와 audience 포함 여부(claims.getAudience().contains(audience))를 검증하는 조건문을 명시적으로 추가함. 또한 프로젝트의 ApiError 클래스 생성자 규격(401, "에러코드", "메시지")에 정확히 맞도록 예외 던지기 구문을 직접 수정·보완함.

검증 방법과 결과: 자동화 테스트 스크립트(check_api.py)를 실행하여 토큰 관련 검증 항목들(PASS no token me, PASS tampered GET, PASS token expired 등)을 포함한 전체 40 개 테스트 항목이 에러 없이 전원 통과(All 40 checks passed)하는 것을 터미널에서 직접 확인하고 검증함.

## 참고 자료

| 자료 제목과 URL | 사용한 위치 | 반영 내용 및 출처 표시 |
|---|---|---|
| 06 주차_JWT_로그인과인증.html | AuthService.java, BoardController.java (및 프론트엔드 API/Vue 파일) | JWT 를 활용한 HS256 토큰 발급(issueToken), Authorization: Bearer 헤더 기반의 엄격한 클레임 및 서명 검증(currentUserId), 작성자 권한 제어(assertOwner) 로직 구현 내용 |

**웹서비스설계 06**

# 이해도 확인과 회고

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

프론트엔드에서 보내준 authorId 를 서버가 아무 의심 없이 그대로 믿고 쓰면, 악의적인 사용자가 개발자 도구(F12)나 Postman 같은 걸로 본인 ID 를 남의 ID 로 바꿔서 요청을 날렸을 때 다른 사람의 글을 몰래 수정하거나 지울 수 있게 됩니다. 이걸 막기 위해 제 코드는 클라이언트가 주는 값을 쓰는 게 아니라, 서버에서 토큰을 까서 인증된 사용자 ID(currentUserId)를 직접 가져오고, assertOwner 함수에서 실제 DB 에 있는 글 작성자 ID 랑 비교해서 다르면 아예 튕겨내보내도록 구현했습니다.

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

화면(프론트엔드)에서 권한이 없다고 수정이나 삭제 버튼을 안 보이게 숨겨두는 건 그냥 보여주기용 UI 조치일 뿐입니다. 마음만 먹으면 API 주소로 직접 요청을 날릴 수 있기 때문에 보안 대책으로는 절대 충분하지 않습니다. 그래서 서버 측에서 확실하게 잡아줘야 하는데, 제 코드에서는 `AuthService.java` 의 `assertOwner` 함수가 로그인한 사람의 ID 랑 글 쓴 사람의 ID 를 비교해서 권한을 확인하고, 다르면 `403 Forbidden` 에러를 내보네 차단합니다.

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

단순 디코드(Decode)는 토큰 안에 든 내용(Payload)을 꺼내서 읽기만 하는 거라 내용이 뭔지는 볼 수 있어도 위조된 건지 알 수가 없습니다. 반면에 검증(Verify)은 서버가 가진 비밀키랑 알고리즘(HS256)을 돌려가면서 이 토큰이 진짜 우리 서버에서 정상적으로 만들어진 게 맞는지 확인하는 과정입니다. 제 코드에서는 currentUserId 함수 안에서 서명이 맞는지 검증하고, 토큰이 만료되지는 않았는지(ExpiredJwtException), 그리고 issuer 랑 audience 클레임까지 다 맞는지 꼼꼼하게 확인하고 있습니다.

4. 로그아웃, 토큰 만료, 새로 고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

**로그아웃**: 브라우저 스토리지 등에 저장되어 있던 토큰을 지워버려서 클라이언트가 인증 정보를 잃어버리게 만듭니다.

**토큰 만료**: 서버가 정해둔 유효기간(exp)이 지나서 파싱할 때 ExpiredJwtException 이 터지면서 서버가 더 이상 그 토큰을 안 받아줍니다.

**새로 고침**: 화면 상태나 컴포넌트는 초기화되지만, 토큰 자체가 로컬에 남아 있다면 로그인 상태는 그대로 유지됩니다.

다만 이미 누군가 토큰을 복사해서 빼돌렸다면, JWT 특성상 서버가 강제로 만료하기 전까지는 유효기간 내에 언제든 남의 계정인 것처럼 쓸 수 있다는 보안상 한계가 있습니다.

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

**원인 확인**: issuer 랑 audience 검증 로직을 짤 때 제일 헤맸습니다. 토큰 안에 들어있는 audience 가 그냥 단순 문자열이 아니라 리스트 형태로 담겨 있어서, 평소처럼 단순 비교를 했더니 자꾸 INVALID_TOKEN 에러가 떨어지는 상황이 발생했습니다.

**수정**: AuthService.java 의 currentUserId 함수에서 단순 비교 대신 .contains(audience)를 써서 리스트 안에 값이 제대로 포함되어 있는지 안전하게 확인하도록 코드를 고쳤습니다. 그리고 예외가 났을 때 프로젝트 규격에 맞는 ApiError 코드가 정확히 나가도록 잡았습니다.

**재검증**: 코드를 고치고 나서 터미널에 python tools/check_api.py 를 돌리고, 토큰 변조나 만료 테스트를 포함해서 40 개 테스트 항목이 전부 PASS(All 40 checks passed)하는 걸 직접 눈으로 확인하고 마무리했습니다.

현재 남은 문제와 다음에 개선할 점: 이번 과제에서 AI 의 도움을 받아 기능과 코드를 효율적으로 구현했지만, 앞으로는 AI 의존도를 줄이고 내 머리로 직접 코드를 고민하며 작성하는 과정을 늘려 스스로의 프로그래밍 문제 해결 능력을 더욱 탄탄히 다지는 것

제출 확인: [✓ ] 실제 코드와 설명 일치 [ ✓] 테스트 직접 수행 [✓ ] AI 활용 및 출처 기록 [ ✓] 비밀 키와 토큰 제거

GitHub 확인: [ ✓] 루트 README 작성 [✓ ] 보고서 PDF 와 증거 업로드 [ ✓] README 문서 링크 확인 [ ✓] 교수자 접근 가능 [ ✓] 저장소 URL 과 커밋 SHA 제출